

Using AI Agents for Data Scraping

Google Trends for your own topic, and getting everyone started

Shihao Yang • Candice Djorno

July 20, 2026

SISMID 2026 • Day 1, 3:30

From one topic to your topic

This morning you scraped Google Trends for **one** topic (dengue in Mexico). This session gets **everyone** comfortable scraping Google Trends for **their own** disease and place, before we add other streams tomorrow.

The goal this session

A reliable, repeatable way to pull Google Trends for whatever you care about, with the agent doing the typing and you keeping the judgment.

- The agent is the **coding tool woven through the whole course**; today we make it second nature on the stream you already know.
- **Plan B** pre-filled notebooks exist, so preferring not to drive an agent never blocks you.

The general recipe

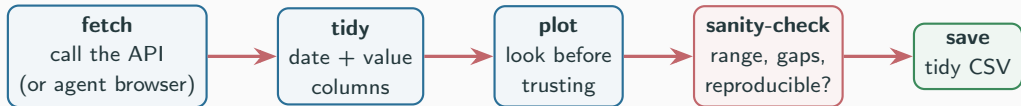
Making the scrape robust

When scraping is blocked: proxy and VPN

Plan B and wrap-up

The general recipe

Every stream, the same five steps



You learned this on Google Trends this morning. It does not change from stream to stream. Only the fetch step differs, and the agent handles that.

Anatomy of a good scrape prompt

Four ingredients: the **outcome**, the **source**, the **shape**, and the **check**.

Template

```
Pull <what series> for <place/topic> over <time window>  
from <source / API>. Return a tidy DataFrame: a date  
column plus one column per series. Then plot it and tell  
me <the check: date range, missing values, does it look  
seasonal?>.
```

The check is not optional. It is what keeps the agent honest and what teaches you to read the output.

Scrape it for your own topic

The whole point: swap in *your* disease and place. One sentence changes.

Prompt (edit the two bold parts)

```
Using pytrends, pull Google Trends for my topic:  
terms about *RSV* in *the United States* (geo=US),  
past 5 years. Tidy date + one column per term, retry on  
429, plot, and print the last date.
```

- Everyone in the room can pull a *different* disease and place at the same time. No one pre-stages a CSV for each.
- This is the “you no longer need a HealthMap” idea: the agent assembles a signal for *your* county and *your* disease on the spot.

Term vs topic (and building a topic from phrases)

Google Trends can give you two different things for “flu”:

- a **term** (the literal string), which misses synonyms, spellings, and other languages, and shows more zeros; or
- a **topic** (Google’s aggregated Knowledge Graph entity), which folds all of those together, with more volume and fewer zeros.

Two ways to reach a topic-like signal with the agent:

Prompt

```
Pull three things for my disease and compare: (1) the individual terms, (2) one combined additive query joining the phrases with " + " (a topic-like OR), and (3) the real Google Trends topic entity (resolve it with pytrends suggestions and pull by its mid). Which has the fewest zeros and the strongest seasonal signal?
```

Verify what the agent produced (still your job)

Fast is not the same as correct. Every scraped stream earns a few cheap checks:

- **Range and gaps:** does the span match what you asked? Missing weeks become wrong models.
- **Scale sanity:** plausible values, not all zeros or a flat line from a failed pull.
- **Reproducibility:** pull twice. Google Trends is noisy, so two pulls can differ; that is the sampling issue from this afternoon.
- **Face validity:** do the peaks land in the right season?

The habit

One plot and one sanity question per stream, before it enters any model.

Making the scrape robust

Google Trends has no official API

Google Trends is itself the messy case: no official API, an internal endpoint that keeps changing, aggressive rate limits, and the sampling instability from this afternoon.

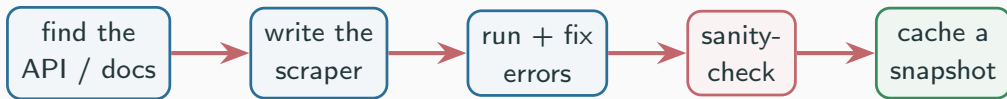
- `pytrends` scrapes that endpoint for you; when it breaks, the agent reads the error, adjusts, and retries, instead of you fighting it.
- You describe the *outcome* (terms, geo, window, retry-on-429); the agent writes the plumbing and hands you a tidy CSV.

The shift

Old way: fight an unofficial API for an afternoon. New way: a paragraph, then you are checking a plot. The hard part becomes **judgment**, not plumbing.

Let the agent run the whole loop

A single, richer prompt can chain the steps and leave you an artifact to inspect:



You stay in the loop at the sanity-check. The agent does the typing and the debugging around it.

When scraping is blocked: proxy
and VPN

Getting blocked is a normal part of scraping

Pull hard enough, from the wrong kind of address, and a server pushes back with a **429** or an outright block. Three tiers, in order:

- **First: retry.** Most 429s clear on a wait-and-retry. We tested Google Trends from a Codespace on an **Azure** IP: 429, wait, then success. Usually enough.
- **Backup: a proxy or VPN.** If retries keep failing (a whole room at once, or a venue whose IP is blocked), route through a machine with an ordinary IP.
- **Last resort: the cache.** Every stream ships a cached snapshot, so no one is ever stuck.

We would rather route around a block than accept it. The cache is the floor, not the plan.

What is a proxy or a VPN?

Every request your computer sends carries a “return address” (its IP). When a site blocks that address, you route the request through *another* machine, so the site sees *its* address instead of yours.

- **Proxy:** reroute just one app (here, the data download).
- **VPN:** reroute *all* of a machine’s traffic the same way.

A spare laptop on ordinary home internet becomes a normal-looking doorway for our cloud Codespaces, whose Azure address some sites throttle.

Same trick a traveler uses to watch home TV abroad. Nothing exotic.

You do not set this up by hand: ask an agent

Standing up a proxy used to be a sysadmin chore. Now you describe the goal in plain words and let a coding agent handle the details:

Prompt (paste into the agent on your spare machine)

```
I'm teaching a class and a website is blocking our shared
cloud computers from downloading data. I have a spare
laptop on normal home internet. Set it up so my students
can send their downloads through this machine, and tell me
in plain steps what to run and exactly what each student
types. Keep it simple and secure.
```

The agent picks the tool, sets a password, and hands you the student line. We already built ours; here is the proof.

We built one, and it works

We stood up exactly this: a spare machine on ordinary home broadband running a small, password-protected proxy (tinyproxy), reachable from anywhere via a tunnel (ngrok).

Proof, from a real Codespace routed through it

```
egress via proxy: a residential IP (home broadband, NOT Azure)
homepage: 200 token: 200 data: 200
SUCCESS: 92 weekly points; last point = this week
VERDICT: Google Trends scraping WORKS via the proxy
```

*A student turns it on with one line: `export
HTTPS_PROXY=http://user:pass@host:port`*

Plan B and wrap-up

Plan B is a first-class path

Not everyone wants to drive an agent, and that is fine.

- Every stream has a **pre-filled notebook** that is a captured example of what the agent produces from these prompts. Run it top to bottom.
- Every stream has a **cached snapshot**, so a failed or blocked live pull is a non-event.
- Pair up: an experienced neighbor plus the agent is the fastest way to learn what good output looks like.

Lane A (agent) and Lane B (notebook) reach the same tidy CSV. Switch any time.

What you can now do

- Scrape **Google Trends** for **your own** disease and place with one prompt, and check the result yourself.
- Make it robust: retry on 429, keep the sampling gotchas in mind.
- Get **around a block** with retry, then a proxy or VPN, with the cache as a floor.
- Fall back to Plan B notebooks whenever you prefer.

The agent is your data-collection layer for the rest of the week.

Looking ahead: Day 2

- **Tomorrow morning:** data beyond Google Trends, Wikipedia pageviews and wastewater in depth, scraped the same way you did today.
- **Then:** mobility and news, and folding every stream into ARGO and the capstone.

You have the method. Day 2 is just more streams through the same loop.

Questions?

Repo: `github.com/Shihao-Yang/sismid2026`